

DSA Pattern Recognition — Interview Battle Guide

Your ChatGPT list is a solid skeleton. What's missing is the stuff that actually wins interviews: **how to recognize the pattern out loud, the exact complexity tradeoffs, the code skeleton you can type from muscle memory, the traps interviewers set, and the follow-up questions they ask once you solve it.**

That's what's added below. Use this as your primary DSA reference — 21 patterns instead of 17, each with a "say this out loud" recognition line (good for your English-communication practice too), a code template, real interview problems, common mistakes, and likely follow-ups.

How to use this in an interview

1. **Restate the problem in your own words** before touching the pattern.
 2. **Say the recognition trigger out loud** — e.g. "This is a continuous subarray with a min/max constraint, so I'm thinking sliding window." Interviewers score you on *process*, not just the final answer.
 3. **State brute force + its complexity first**, then say why you're optimizing. This shows range and protects you if the optimal solution stalls halfway.
 4. **Code the template, then adapt.** Don't derive two-pointer/sliding-window boilerplate from scratch every time — that's where candidates burn clock.
 5. **Always state final time and space complexity unprompted.** Interviewers almost never have to ask for it if you're strong — that itself is a signal.
-

1. Brute Force

Recognition line: *"I don't see structure yet, let me start with brute force to make sure I understand the problem correctly, then optimize."*

When to use: Always your first 30 seconds of thinking, even if you know the optimal pattern — verbalizing brute force shows problem-solving process.

Keywords: every, all, generate, count all

Complexity: $O(n^2)$, $O(n^3)$, $O(2^n)$

Interviewer follow-up to expect: *"Can you do better than $O(n^2)$?"* — have your optimization already half-planned before they ask.

2. Two Pointers

Recognition line: *"The array is sorted / I need pairs from both ends, so two pointers gets me $O(n)$ instead of $O(n^2)$."*

When to use: Sorted array, palindrome check, pair-sum, merging, removing duplicates in place.

Keywords: sorted, pair, closest, palindrome, in-place

Template:

javascript

```
let left = 0, right = arr.length - 1;
while (left < right) {
  const sum = arr[left] + arr[right];
  if (sum === target) return [left, right];
  else if (sum < target) left++;
  else right--;
}
```

Interview problems: Two Sum II, 3Sum, Container With Most Water, Valid Palindrome, Trapping Rain Water (two-pointer variant), Sort Colors (Dutch National Flag — see pattern 21)

Common mistake: Forgetting the array must be sorted first, or using two pointers on an unsorted problem where a HashMap is actually correct (plain Two Sum on unsorted array — that's HashMap, not two pointers).

Complexity: $O(n)$ time, $O(1)$ space — call this out, it's the main selling point over brute force $O(n^2)$.

3. Sliding Window ★★★★★

Recognition line: *"This is a contiguous subarray/substring problem where I can reuse work from the previous window instead of recomputing — sliding window."*

When to use: Continuous subarray or substring with a size/sum/distinct constraint.

Keywords: longest, shortest, maximum, minimum, at most, exactly K, window, continuous, substring, subarray

Two flavors — say which one you're using:

- **Fixed size window** (size K given) — simple slide.
- **Variable size window** (grow until invalid, shrink until valid again) — this is the one interviewers love because candidates fumble the shrink logic.

Template (variable window):

javascript

```
let left = 0, result = 0, windowState = new Map();
for (let right = 0; right < arr.length; right++) {
  // expand: add arr[right] to windowState
  while (/* window invalid */) {
    // shrink: remove arr[left] from windowState
    left++;
  }
  result = Math.max(result, right - left + 1);
}
```

Interview problems: Longest Substring Without Repeating Characters, Minimum Window Substring, Fruit Into Baskets, Subarrays with K Distinct Integers, Maximum Average Subarray, Longest Repeating Character Replacement, Sliding Window Maximum (needs monotonic deque — see pattern 15)

Common mistake: Using $>$ instead of $\geq$ in the shrink condition, or recomputing the window state from scratch instead of incrementally updating it (defeats the whole purpose — $O(n \cdot k)$ instead of $O(n)$).

Follow-up interviewers ask: "What if K distinct became exactly K , not at most K ?" — know the " $\text{atMost}(K) - \text{atMost}(K-1)$ " trick for exact- K variants.

4. Nested Loop + Frequency/Contribution Map

Recognition line: "Every substring/subarray contributes to the final answer, so there's no invalid state to shrink away from — this isn't sliding window, it's a contribution-counting problem."

When to use: Every substring/subarray matters to the sum, not just the longest/shortest one.

Keywords: beauty sum, count all substrings, sum of all subarrays

Interview problems: Sum of Beauty of All Substrings, Count Beautiful Substrings, Count Vowel Substrings, Sum of Subarray Minimums (usually solved better with a monotonic stack for $O(n)$ — mention this upgrade path)

Why interviewers ask this: it's the classic trap where candidates mistakenly reach for sliding window and get stuck because there's no shrink condition. Naming *why* it's not sliding window is a strong signal.

5. Prefix Sum (and Prefix Sum + HashMap)

Recognition line: "I need repeated range queries, so I'll precompute prefix sums once for $O(1)$ range lookups instead of $O(n)$ per query."

When to use: Repeated range-sum queries, or subarray-sum-equals-K type problems.

Formula: $\text{sum}(L, R) = \text{prefix}[R] - \text{prefix}[L-1]$

Upgrade — Prefix Sum + HashMap: when you need to count subarrays with $\text{sum} == K$ (not just answer one query), store prefix sums seen so far in a HashMap.

Template:

javascript

```
let prefixSum = 0, count = 0;
const seen = new Map([[0, 1]]);
for (const num of arr) {
  prefixSum += num;
  count += seen.get(prefixSum - k) || 0;
  seen.set(prefixSum, (seen.get(prefixSum) || 0) + 1);
}
```

Interview problems: Range Sum Query — Immutable, Subarray Sum Equals K, Continuous Subarray Sum, Product of Array Except Self (prefix product variant)

Common mistake: Off-by-one on $[L-1]$, and forgetting to seed the map with $\{0: 1\}$ for subarrays starting at index 0.

6. HashMap / HashSet

Recognition line: "I need $O(1)$ lookup for duplicates/frequency/existence — trading space for time with a hash map."

Keywords: duplicate, frequency, exist, visited, lookup, anagram

Interview problems: Two Sum, Group Anagrams, Happy Number, Longest Consecutive Sequence, Contains Duplicate II, Isomorphic Strings

Follow-up interviewers ask: "Can you do it in $O(1)$ space?" — usually means sacrificing the original array or using the input itself as a hash (e.g. negation marking, or index-as-hash tricks) — know at least one example.

7. Binary Search (and Binary Search on Answer)

Recognition line: "The search space is monotonic — if X works, $X+1$ also works — so I can binary search even though there's no explicit sorted array."

Keywords: sorted, minimum, maximum, search, first, last, threshold

The upgrade that impresses interviewers — Binary Search on the Answer: used when the array itself isn't sorted, but the *answer space* is monotonic (true/false flips once as you increase a candidate value).

Template (search on answer):

```
javascript
```

```
let lo = minPossible, hi = maxPossible, ans = hi;
while (lo <= hi) {
  const mid = lo + Math.floor((hi - lo) / 2);
  if (isFeasible(mid)) { ans = mid; hi = mid - 1; }
  else lo = mid + 1;
}
```

Interview problems: Search Insert Position, Capacity To Ship Packages Within D Days, Koko Eating Bananas, Split Array Largest Sum, Find Minimum in Rotated Sorted Array, Median of Two Sorted Arrays (hard, FAANG favorite)

Common mistake: $\text{mid} = (\text{lo} + \text{hi}) / 2$ causing integer overflow in other languages (less of an issue in JS, but say it anyway — shows rigor), and infinite loops from wrong lo/hi updates when boundaries are inclusive vs exclusive.

8. Expand Around Center

Recognition line: "Palindromes are symmetric around a center, so instead of checking every substring $O(n^2)$ I expand outward from each possible center — $O(n^2)$ time but $O(1)$ space, cleaner than DP"

Keywords: palindrome, longest palindrome

Interview problems: Longest Palindromic Substring, Palindromic Substrings (count all)

Follow-up: "Can you do this in $O(n)$?" — mention Manacher's Algorithm by name even if you don't fully implement it; naming it signals depth.

9. Dynamic Programming

Recognition line: "The current answer depends on overlapping smaller subproblems, and there's optimal substructure — so I'll define state, a recurrence, and either memoize top-down or tabulate bottom-up."

Keywords: minimum cost, maximum profit, ways, count, optimal

The 4-step framework — say these out loud in order:

1. Define the state (what does $\text{dp}[i]$ mean?)

2. Find the recurrence relation
3. Identify base cases
4. Decide iteration order (and whether you can compress space to $O(1)$ or $O(n)$ instead of $O(n^2)$)

Interview problems: House Robber, Coin Change, Longest Increasing Subsequence, Longest Common Subsequence, 0/1 Knapsack, Edit Distance, Unique Paths, Partition Equal Subset Sum

Common mistake: Jumping straight to code without stating the recurrence — interviewers often stop candidates here because the recurrence *is* the solution; the code is just translation.

Follow-up interviewers ask: "Can you reduce the space complexity?" — always have the space-optimized rolling-array version ready for 1D/2D DP.

10. Backtracking

Recognition line: "I need to explore all valid combinations and prune invalid branches early — backtracking with a choose/explore/unchoose loop."

Keywords: generate, combination, permutation, subset, N-Queens, Sudoku

Template:

```
javascript
```

```
function backtrack(path, choices) {  
  if (isComplete(path)) { result.push([...path]); return; }  
  for (const choice of choices) {  
    if (!isValid(choice)) continue;  
    path.push(choice);           // choose  
    backtrack(path, nextChoices); // explore  
    path.pop();                  // un-choose  
  }  
}
```

Interview problems: Subsets, Permutations, Combination Sum, N-Queens, Word Search, Palindrome Partitioning, Generate Parentheses

Common mistake: Forgetting the "un-choose" step (mutating shared state without backtracking it), and not pruning early which turns an acceptable solution into a TLE.

11. BFS

Recognition line: "I need the shortest path in an unweighted graph or grid — BFS"

guarantees shortest path because it explores level by level."

Keywords: minimum steps, shortest path, nearest, level order

Interview problems: Rotten Oranges, Word Ladder, Binary Tree Level Order Traversal, 01 Matrix, Shortest Path in Binary Matrix, Number of Islands (can also be BFS)

Common mistake: Using BFS on a *weighted* graph expecting shortest path — that needs Dijkstra, not plain BFS. Know this distinction; it's a common verbal trap question.

12. DFS

Recognition line: *"I need to explore full paths/connectivity, not shortest distance — DFS, recursive or with an explicit stack."*

Keywords: connected, island, tree, explore, path exists

Interview problems: Number of Islands, Clone Graph, Path Sum, Course Schedule (cycle detection), Surrounded Regions

Follow-up: *"Can you do this iteratively instead of recursively?"* — know how to convert DFS recursion into an explicit stack to avoid call-stack overflow on deep inputs — this comes up a lot in senior-level rounds.

13. Heap / Priority Queue

Recognition line: *"I repeatedly need the current smallest/largest element as the data changes — a heap gives me $O(\log n)$ insert/extract instead of $O(n)$ re-sorting."*

Keywords: top K, largest, smallest, median, kth

Interview problems: Kth Largest Element, Merge K Sorted Lists, Top K Frequent Elements, Find Median from Data Stream (two-heap technique — see pattern 19), Task Scheduler

Common mistake: Using a max-heap when you only need the top K smallest (you actually want a max-heap of size K, popping the largest each time to keep the heap small) — get the heap direction right, interviewers probe this.

14. Greedy

Recognition line: *"A locally optimal choice at each step leads to a globally optimal result here — I need to justify why greedy works before coding it."*

Keywords: minimum, maximum, interval, schedule, activity selection

The trap: greedy looks like it should always work but often doesn't — always verify with a counterexample mentally, or explicitly state the exchange argument/proof sketch for why

greedy is safe here. This single habit separates strong candidates from average ones.

Interview problems: Jump Game, Gas Station, Merge Intervals (also its own pattern, see below), Non-overlapping Intervals, Meeting Rooms II

15. Monotonic Stack

Recognition line: *"I need the next/previous greater or smaller element efficiently — a monotonic stack lets me do this in $O(n)$ instead of $O(n^2)$ brute force."*

Keywords: next greater, previous smaller, histogram, span

Template:

javascript

```
const stack = []; // indices, kept increasing (or decreasing) by value
const result = new Array(arr.length).fill(-1);
for (let i = 0; i < arr.length; i++) {
    while (stack.length && arr[i] > arr[stack[stack.length - 1]]) {
        const idx = stack.pop();
        result[idx] = arr[i];
    }
    stack.push(i);
}
```

Interview problems: Daily Temperatures, Largest Rectangle in Histogram, Trapping Rain Water (stack variant), Next Greater Element I & II, Sum of Subarray Minimums, Stock Span Problem

Same idea, sliding window flavor — Monotonic Deque: used for Sliding Window Maximum/Minimum in $O(n)$. Mention this as the natural upgrade when someone asks for $O(n)$ window-max instead of $O(nk)$ with a heap.

16. Trie

Recognition line: *"I'm doing repeated prefix lookups across many words — a trie gives me $O(L)$ per operation where L is word length, instead of scanning the whole dictionary."*

Keywords: prefix, autocomplete, dictionary, word search

Interview problems: Implement Trie (Prefix Tree), Word Search II, Design Add and Search Words Data Structure, Longest Word in Dictionary

17. Union Find (Disjoint Set)

Recognition line: *"I need to dynamically track connected components and answer 'are these connected?' queries efficiently — Union Find with path compression and union by rank gives near- $O(1)$ amortized operations."*

Keywords: connected, network, groups, redundant edge, provinces

Interview problems: Number of Provinces, Redundant Connection, Accounts Merge, Number of Islands II, Graph Valid Tree

Mention unprompted: path compression + union by rank/size — without both, Union Find degrades toward $O(n)$; with both it's nearly $O(1)$ amortized (inverse Ackermann). Naming this is a strong senior-level signal.

18. Merge Intervals

Recognition line: *"These are ranges that might overlap — sort by start time first, then sweep and merge."*

Keywords: overlapping, intervals, schedule, merge, insert interval

Template:

javascript

```
intervals.sort((a, b) => a[0] - b[0]);
const merged = [intervals[0]];
for (const [start, end] of intervals.slice(1)) {
  const last = merged[merged.length - 1];
  if (start <= last[1]) last[1] = Math.max(last[1], end);
  else merged.push([start, end]);
}
```

Interview problems: Merge Intervals, Insert Interval, Non-overlapping Intervals, Meeting Rooms I & II, Employee Free Time

19. Fast & Slow Pointers (Floyd's Cycle Detection)

Recognition line: *"I need to detect a cycle or find a midpoint in a linked list without extra space — two pointers moving at different speeds."*

Keywords: cycle, linked list, middle, duplicate number, happy number

Interview problems: Linked List Cycle I & II, Middle of the Linked List, Find the Duplicate Number, Happy Number, Palindrome Linked List

Follow-up: *"How do you find where the cycle starts, not just whether one exists?"* — know the two-phase Floyd's algorithm (reset one pointer to head, move both at same speed, they

meet at cycle start).

20. Two Heaps (for running median / balancing)

Recognition line: *"I need to maintain a running median or balance two halves of a stream — a max-heap for the lower half and a min-heap for the upper half."*

Interview problems: Find Median from Data Stream, Sliding Window Median, IPO (Maximize Capital)

21. Cyclic Sort / Dutch National Flag

Recognition line: *"The array contains numbers in a known range $1..n$ — I can place each number at its correct index in one pass instead of sorting, $O(n)$ time $O(1)$ space."*

Interview problems: Missing Number, Find All Duplicates in an Array, First Missing Positive, Sort Colors (Dutch National Flag — 3-way partition with low/mid/high pointers)

22. Bit Manipulation

Recognition line: *"This is asking about XOR/AND/OR properties or wants $O(1)$ space — bit tricks."*

Keywords: XOR, single number, power of two, subsets via bitmask, count bits

Interview problems: Single Number I/II/III, Counting Bits, Sum of Two Integers (no + operator), Subsets (bitmask enumeration as an alternative to backtracking — good to mention both approaches)

Know cold: $(n \& (n-1))$ clears the lowest set bit — used in Number of 1 Bits and Power of Two checks.

Additional patterns worth knowing by name (lower frequency but do come up)

Pattern	One-line trigger
Topological Sort	Ordering with dependencies — "prerequisite" problems
Kadane's Algorithm	Maximum subarray sum, a specialized 1D DP
K-way Merge	Merging K sorted lists/arrays — usually heap-based

Pattern	One-line trigger
Divide and Conquer	Problem splits into independent halves — merge sort, quickselect
Segment Tree / Fenwick Tree	Range query + range update, both needed — mention when prefix sum isn't enough because of updates
Matrix Traversal (spiral, rotate)	Grid geometry problems, in-place rotation
Meet in the Middle	Exponential search space split in half — subset-sum on larger N

Master Decision Flowchart

text

Linked List?

Cycle/middle? -> Fast & Slow Pointers

Reverse in place? -> In-place reversal (two/three pointer swap)

Array / String?

Sorted?

Pair sum? -> Two Pointers

Search/threshold? -> Binary Search (or Binary Search on Answer)

Contiguous subarray/substring?

Longest/shortest/at-most K? -> Sliding Window

Every substring contributes? -> Nested Loop + Frequency Map

Repeated range sum queries? -> Prefix Sum (+ HashMap if counting)

Range 1..n, in-place O(1) space? -> Cyclic Sort / Dutch Flag

Next greater/smaller element? -> Monotonic Stack

Palindrome? -> Expand Around Center

Need frequency/duplicate/lookup? -> HashMap / HashSet

XOR / bit tricks / O(1) space? -> Bit Manipulation

Intervals? -> Merge Intervals / Greedy

Generate all combinations/permutations? -> Backtracking

Top K / running median / streaming? -> Heap or Two Heaps

Tree? -> DFS (paths) / BFS (level order)

Graph?

Shortest path, unweighted? -> BFS

Full traversal/connectivity? -> DFS

Dynamic connectivity queries? -> Union Find

Dependency ordering? -> Topological Sort

Prefix/dictionary lookups? -> Trie

Optimization with overlapping subproblems? -> Dynamic Programming

Optimization with provable local choice? -> Greedy

Recommended mastery order (builds on your Striver roadmap)

1. Arrays + HashMap/HashSet
2. Two Pointers
3. Sliding Window
4. Prefix Sum
5. Binary Search (+ Binary Search on Answer)

6. Stack & Queue → Monotonic Stack
 7. Linked List → Fast & Slow Pointers
 8. Trees (DFS/BFS)
 9. Heap / Two Heaps
 10. Intervals / Merge Intervals
 11. Graphs → BFS/DFS → Union Find → Topological Sort
 12. Backtracking
 13. Dynamic Programming
 14. Tries
 15. Greedy
 16. Bit Manipulation
 17. Cyclic Sort, Segment Tree, K-way Merge — pick up as needed per company
-

Interview execution checklist (say these out loud, in order)

1. Restate the problem and confirm constraints (array size, value range, sorted or not, duplicates allowed?) — this alone catches ambiguity early.
2. State brute force + complexity.
3. Name the pattern and *why* ("recognition line" style above).
4. Code the skeleton, narrate as you type.
5. Trace through one example by hand before saying "I think this works."
6. State final time and space complexity unprompted.
7. Proactively mention edge cases: empty input, single element, all duplicates, negative numbers, integer overflow (less relevant in JS but mention it — shows rigor).
8. If interviewer asks "can you optimize further" and you're already optimal, say so and explain *why* it's optimal (e.g. "we have to look at every element at least once, so $O(n)$ is a hard lower bound here").

This checklist matters as much as knowing the pattern — interviewers at Amazon, Grab, and most Gulf-market product companies score communication and process alongside correctness, especially at Senior/Lead level where you're expected to narrate tradeoffs the way you would in a design review.